# The Complete Flow — Every Step in Detail

> From the moment you click **Send** in the chat UI to the moment the answer appears, here is exactly what happens: where control goes, what work is done, and how the backend produces the answer.

---

## Table of Contents


1. The two user actions
2. Flow A — File upload (click-by-click)
3. Flow B — Ask a question (click-by-click)
4. The HTTP requests and responses (exact payloads)
5. Every error path, clearly
6. Full end-to-end sequence diagram


---

## 1. The two user actions

The app has exactly **two buttons** that trigger the backend:

| Button | What the user does | Which function it calls |
|---|---|---|
| **Upload** | Selects a file, clicks Upload | `process-file` |
| **Send** | Types a question, clicks Send | `ask-question` |

Everything below traces control for both.

---

# 2. Flow A — File upload (click-by-click)

## Step-by-step control flow

```
│    • receives {stored: n}                                                  │
│    • shows "Uploaded successfully (n chunks stored)"                       │
│    • stops the spinner                                                     │
└────────────────────────────────────────────────────────────────────────────┘
```

## What the `documents` table looks like after upload

```
┌──────────────────┬──────────────────────────────┬──────────────┬──────────────┬──────────────────────┐
│ id (uuid)        │ content (text)               │ embedding    │ file_name    │ created_at (timestamp) │
├──────────────────┼──────────────────────────────┼──────────────┼──────────────┼──────────────────────┤
│ 3f1a...9e2b      │ "Vicky is a friendly cat..." │ [0.12,-0.34,...]│ notes.txt    │ 2026-08-05 10:02:11  │
│ 7c9b...11d4      │ "She likes to sleep..."      │ [-0.01,0.88,...]│ notes.txt    │ 2026-08-05 10:02:11  │
│ a2e4...55f0      │ "Tax rules for 2026..."      │ [0.45,0.02,...] │ report.pdf   │ 2026-08-05 10:05:33  │
└──────────────────┴──────────────────────────────┴──────────────┴──────────────┴──────────────────────┘
```

> Every chunk = one row. The `embedding` column (768 numbers) is what makes these rows searchable **by meaning**.

---

# 3. Flow B — Ask a question (click-by-click)

## Step-by-step control flow

```
| 8. ask-question — stage 5: BUILD THE CONTEXT  (buildPrompt)                |
|    • take the 3 chunk texts (row.content)                                  |
|    • join them with a blank line between each                              |
|    • assemble the prompt:                                                  |
|        "You are an assistant that answers questions using ONLY the         |
|         provided context.                                                  |
|                                                                            |
|         Context:                                                           |
|         <chunk 1 text>                                                     |
|                                                                            |
|         <chunk 2 text>                                                     |
|                                                                            |
|         <chunk 3 text>                                                     |
|                                                                            |
|         Question: who is Vicky?                                            |
|                                                                            |
|         Answer using only the context above. If the context does not       |
|         contain the answer, say that you don't know."                      |
└────────────────────────────────────────────────────────────────────────────┘
                                  |
                                  ▼
┌────────────────────────────────────────────────────────────────────────────┐
| 9. ask-question — stage 6: GENERATE THE ANSWER  (generateAnswer)           |
|    • HTTP POST to Gemini:                                                  |
|         models/gemini-flash-latest:generateContent                         |
|         body: { contents: [{ parts: [{ text: "<the prompt>" }] }],         |
|                 generationConfig: { temperature: 0.4 } }                   |
|    • Gemini reads ONLY the context in the prompt                           |
|    • Gemini replies with the answer text                                   |
|    • empty reply? → return empty string                                    |
└────────────────────────────────────────────────────────────────────────────┘
                                  |
                                  ▼
┌────────────────────────────────────────────────────────────────────────────┐
| 10. ask-question — stage 7: REPLY TO THE FRONTEND                          |
|    • respond: 200 OK  { "answer": "Vicky is a friendly cat who likes..." }  |
└────────────────────────────────────────────────────────────────────────────┘
                                  |
                                  ▼
┌────────────────────────────────────────────────────────────────────────────┐
| 11. FLUTTER APP:                                                           |
|    • receives {answer: "..."}                                              |
|    • stops the typing indicator                                            |
|    • renders the answer in a chat bubble                                   |
|    • scrolls to the bottom                                                 |
└────────────────────────────────────────────────────────────────────────────┘
```

# 4. The HTTP requests and responses (exact payloads)

## Request 1 — File upload

```
POST https://<your-project>.supabase.co/functions/v1/process-file
Headers:
  Content-Type: application/json

Body:
{
  "content": "Vicky is a friendly cat. She likes to sleep all day and eat treats.",
  "file_name": "notes.txt"
}
```

```
Response 200:
{
  "stored": 2
}
```

## Request 2 — Ask a question

```
POST https://<your-project>.supabase.co/functions/v1/ask-question
Headers:
  Content-Type: application/json

Body:
{
  "question": "who is Vicky?"
}
```

```
Response 200:
{
  "answer": "Vicky is a friendly cat who likes to sleep all day and eat treats."
}
```

---

## What the Edge Function sends to Gemini (hidden from the user)

### Embedding call (from both functions):

```
POST https://generativelanguage.googleapis.com/v1beta/models/gemini-embedding-001:embedContent?key=GEMINI_API_KEY
Body:
{
  "model": "models/gemini-embedding-001",
  "content": { "parts": [{ "text": "who is Vicky?" }] },
  "outputDimensionality": 768
}

Response:
{ "embedding": { "values": [0.05, -0.42, 0.18, ... 768 numbers] } }
```

### Generation call (from ask-question only):

```
POST https://generativelanguage.googleapis.com/v1beta/models/gemini-flash-latest:generateContent?key=GEMINI_API_KEY
Body:
{
  "contents": [{ "parts": [{ "text": "<the grounded prompt>" }] }],
  "generationConfig": { "temperature": 0.4 }
}

Response:
{ "candidates": [{ "content": { "parts": [{ "text": "Vicky is a friendly cat..." }] } }] }
```

---

## 5. Every error path, clearly

| Where it fails | What happens | Response the user sees |
|---|---|---|
| `content` or `file_name` missing / empty | `process-file` returns 400 | `{"error": "content and file_name are required"}` |
| Text produced no chunks | `process-file` returns 400 | `{"error": "content produced no chunks"}` |
| `GEMINI_API_KEY` not set | both functions throw | 500 `{"error": "GEMINI_API_KEY is not set"}` |
| Gemini embedding API error | both functions throw | 500 `{"error": "Embedding API error <status>: <detail>"}` |
| Embedding has wrong size | both functions throw | 500 `{"error": "Unexpected embedding dimension: <n>"}` |
| Database insert fails | `process-file` throws | 500 `{"error": "<postgres error>"}` |
| `question` missing / empty | `ask-question` returns 400 | `{"error": "question is required"}` |
| No similar chunks found | `ask-question` responds normally | `{"answer": "No relevant context was found. Upload a document first."}` |
| Gemini generation error | `ask-question` throws | 500 `{"error": "Generation API error <status>: <detail>"}` |

## 6. Full end-to-end sequence diagram

```
sequenceDiagram
    autonumber
    participant User
    participant Flutter as Flutter Web App
    participant Sup as Supabase Platform (router)
    participant PF as process-file (Edge Function)
    participant AQ as ask-question (Edge Function)
    participant GE as Gemini Embedding API
    participant GG as Gemini Generation API
    participant DB as Postgres + pgvector (documents table + match_documents)

    Note over User,DB: ═══ PART 1: FILE UPLOAD ═══
    User->>Flutter: clicks Upload (file selected)
    Flutter->>Sup: POST /functions/v1/process-file {content, file_name}
    Sup->>PF: route request → run Deno.serve handler
    PF->>PF: validate content & file_name (400 if bad)
    PF->>PF: chunkText() → split text into ~2000-char pieces
    loop each chunk
        PF->>GE: POST gemini-embedding-001:embedContent(chunk)
        GE-->>PF: 768-dimension vector
    end
    PF->>DB: INSERT rows {content, embedding, file_name}
    DB-->>PF: inserted (n rows)
    PF-->>Sup: 200 {stored: n}
    Sup-->>Flutter: 200 {stored: n}
    Flutter-->>User: "Uploaded successfully (n chunks)"

    Note over User,DB: ═══ PART 2: ASK A QUESTION ═══
    User->>Flutter: types "who is Vicky?" → clicks Send
    Flutter->>Sup: POST /functions/v1/ask-question {question}
    Sup->>AQ: route request → run Deno.serve handler
    AQ->>AQ: validate question (400 if empty)
    AQ->>GE: POST gemini-embedding-001:embedContent("who is Vicky?")
    GE-->>AQ: 768-dimension question vector
    AQ->>DB: RPC match_documents(query_embedding, match_count=3)
    Note over DB: SELECT ... ORDER BY embedding <=> query_embedding LIMIT 3
    DB-->>AQ: top 3 closest chunks {id, content, file_name, similarity}
    alt 0 chunks returned
        AQ-->>Flutter: 200 {answer: "No relevant context found..."}
        Flutter-->>User: shows the message bubble
    else chunks found
        AQ->>AQ: buildPrompt(context, question) → grounded prompt
        AQ->>GG: POST gemini-flash-latest:generateContent(prompt)
        Note over GG: reads ONLY the context in the prompt
        GG-->>AQ: answer text
        AQ-->>Sup: 200 {answer: "Vicky is a friendly cat..."}
        Sup-->>Flutter: 200 {answer: "..."}
        Flutter-->>User: answer shown in chat bubble
    end
```

---

### The 3 key ideas to remember

1. **Every piece of text becomes numbers** — files *and* questions are turned into 768-number vectors by the *same* Gemini embedding model, so they live in the same "meaning space".
2. **Search = find the closest numbers** — `match_documents` ranks all stored rows by cosine distance to the question's vector and keeps the top 3.
3. **The answer is grounded** — only those 3 chunk texts go into the prompt, and Gemini is explicitly told to answer *only* from them (and to say "I don't know" otherwise). That's what makes the answer trustworthy.